



COS 132 - Imperative Programming

Study Unit 6: Control Structures - Repetition Statements

Copyright ©2024 by Linda Marshall, Inge Odendaal and Mark Botros. All rights reserved.

Contents

6.1	Introduction	2
6.2	Recap of Increment and Decrement Operators	2
6.3	for Loops	3
6.3.1	Parts of the for Loop	3
6.3.2	For Loops with Multiple Control Variables	5
6.3.2.1	Using Multiple Variables in for Loops	5
6.3.2.2	Simultaneous Variable Updates at Different Rates	5
6.4	while Loops	6
6.5	do-while Loops	9
6.6	Coding Patterns	10
6.7	Translation between Repetition Statements	18
6.7.1	Translating for Loops to while and do-while Loops	18
6.7.1.1	Using a while Loop	18
6.7.1.2	Using a do-while Loop	18
6.7.2	Translating while and do-while Loops to for Loops	19
6.7.2.1	From while Loop	19
6.7.2.2	From do-while Loop	19
6.8	Good programming practices	20
6.8.1	Infinite Loops	20
6.8.1.1	Loops that look like they are infinite	21
6.8.1.2	Loops not intended to be infinite	21
6.8.1.3	Intentional infinite loops	22
6.8.2	Using break and continue	22
6.8.3	Loop Invariant	23
6.9	Document Change Log	25

6.1 Introduction

Repetition statements (loops) execute specific statements a number of times. This can either be a fixed number or a variable number, based on a condition at runtime. C++ provides several types of loops for handling repetitive tasks: for loops for fixed repetitions, and while and do-while loops for variable repetitions.

In this study unit we will quickly discuss the increments and decrement operators which are used extensively to control loops, especially loops that iterate a fixed number of times. We will then cover the loops mentioned before considering specific patterns (plans) that can be applied to loops, the translation between the different types of loops and programming practices for the application of loops.

6.2 Recap of Increment and Decrement Operators

This section should be read in conjunction with Study Unit 3 where operators were covered in depth. Due to the increment and decrement operators being used to control the indexes of loops, it is good to know where they fit into the precedence hierarchy.

It should be noted that the postfix increment and decrement operators have higher precedence than their prefix counterparts. Therefore the operator precedence hierarchy given in Study Unit 3 is updated as follows.

	Operator	Associativity
0	postfix ++, postfix --	→
1	!, unary -, unary +, prefix ++, prefix --	←
2	/, *, %	→
3	+, -,	→
4	<, <=, >, >=	→
5	==, !=	→
6	&&	→
7		→
8	=, +=	←

Table 1: Updated Operator Precedence to Include Increment and Decrement

Unfortunately, the result of expressions using the increment and decrement operators are dependent on the compiler being used and whether the compiler is implemented using left-to-right or right-to-left evaluation.

Assuming that the compiler makes use of left-to-right evaluation, the result of the statement `i-- + ++j`, for `i = 10` and `j = 20`, will evaluate to 31. That is, `++j` will evaluate first to 21, the current value of `i` will be added to 21, giving a final sum of 31. Only after the addition will `i` be decremented by 1 to give 9. So the output of the following program:

```
#include <iostream>

using namespace std;
int main() {
```

```

    int i = 10;
    int j = 20;

    cout << "i = " << i << " j = " << j << endl;
    cout << i-- + ++j << endl;
    cout << "i = " << i << " j = " << j << endl;

    return 0;
}

```

will be

```

i = 10 j = 20
31
i = 9 j = 21

```

To enforce specific operator ordering, parentheses should be used.

Thankfully, when considering loops, the increment and decrement operators are rarely used as part of complex expressions. They are mostly used to represent the loop counter. The compiler evaluation preference is therefore not a factor.

6.3 for Loops

A for loop is a fixed-repetition statement that is controlled by an index. This loop iterates over a block of code a specific number of times determined by the start and end values of the index. This means that the index of the for loop may be used within the statements that are to be repeated a fixed number of times. It is especially useful when the number of iterations is known before entering the loop. A for loop is structured into three main parts: the initialisation, the condition, and the update parts. Each part plays a crucial role in how the loop operates.

6.3.1 Parts of the for Loop

1. **Initialisation:** This is the first part of the for loop. Here, you can declare and initialise one or more loop control variables. This part executes only once when the loop starts. The variables declared in this part are local to the loop and cannot be accessed outside the loop's scope.
2. **Condition:** This expression is evaluated before each iteration of the loop. As long as this condition evaluates to true, the loop continues to execute. If the condition evaluates to false, the loop stops. This part is important to prevent infinite loops and to ensure that the loop executes the correct number of times.
3. **Update:** This part is executed at the end of each iteration. It is typically used to update the loop control variables. This could involve incrementing or decrementing a counter, or any other operation that moves towards making the condition false to terminate the loop.

The syntax of a for loop in C++ is as follows:

```
for (initialisation; condition; update) {  
    // Code to execute during each iteration  
}
```

Listing 6.1: For Loop Syntax

The following expressions are valid:

- **initialisation:** You can declare new variables or use existing variables. Multiple variables can be declared separated by commas.
- **Condition:** Any expression that evaluates to a boolean value (true or false). It often involves comparison operators.
- **Update:** Any valid expression that modifies the variables declared in the initialisation part. Commonly, it involves arithmetic operations such as increment (++), decrement (--), or any other assignment operation. Refer to following listings.

```
for (index = start_value; index <= end_value; index++) {  
    // statement or block of statements  
}  
  
for (index = start_value; index >= end_value; index--) {  
    // statement or block of statements  
}
```

Listing 6.2: Increasing or decreasing index:

To print the numbers from 2 to 10, the following C++ program uses a for loop:

```
#include <iostream>  
using namespace std;  
  
int main() {  
    for (int i = 2; i <= 10; i++) {  
        cout << i << endl;  
    }  
    return 0;  
}
```

Listing 6.3: For Loop Example

Compound assignment operators are often used as the Update Condition

```
#include <iostream>  
using namespace std;  
  
int main() {  
    int i = 10;  
    for (i; i >= 1; i-=2) {  
        cout << i << endl;  
    }  
}
```

```

    }
    return 0;
}

```

Listing 6.4: Compound Assignment Operator for Update

Question 6.1

Modify the program in Listing 6.3 to count from 10 down to 2.

Question 6.2

Write a program that uses a for loop to print the powers of two from 2^1 to 2^x where x is user defined. Use the `*=` compound assignment operator.

6.3.2 For Loops with Multiple Control Variables

For loops can manage multiple variables within their structure by initialising and updating multiple variables simultaneously.

6.3.2.1 Using Multiple Variables in for Loops

A for loop can initialise and update multiple variables by separating them with commas. This approach is handy when the logic of the loop requires simultaneous updates to these variables, such as converging two indices toward the center of an array or synchronising multiple counters.

The following example demonstrates a for loop that initialises two variables, `i` and `j`. The variable `i` increments from 1 upwards, and `j` decrements from 10 downwards. The loop continues until `i` is less than `j`, effectively demonstrating a meet-in-the-middle strategy which is often used in search algorithms or when pairing elements from opposite ends of a collection.

```

#include <iostream>
using namespace std;

int main() {
    // Initialising two variables in a for loop
    for (int i = 1, j = 10; i < j; i++, j--) {
        cout << "i = " << i << ", j = " << j << endl;
    }
    return 0;
}

```

Listing 6.5: for Loop with Multiple Control Variables

This loop outputs the values of `i` and `j` as they converge towards each other. For loops can also manage more complex updates

6.3.2.2 Simultaneous Variable Updates at Different Rates

In the following example, a for loop is used with two loop variables, `i` and `k`. The variable `i` increments in every iteration, serving as a straightforward loop counter. In contrast, the

variable `k` is designed to increment every second iteration, demonstrating how different update rates can be managed within the same loop structure.

```
#include <iostream>
using namespace std;

int main() {
    // Initialise 'i' for regular increment
    // and 'k' for staggered increment
    for (int i = 0, k = 0; i < 10;
         i++, k += (i % 2 == 0 ? 1 : 0)) {
        cout << "i_=" << i << ",_k_=" << k << endl;
    }
    return 0;
}
```

Listing 6.6: `for` Loop with Staggered Variable Updates

In this code, `i` acts as a regular counter from 0 to 9. The variable `k` increases by 1 every time `i` is even (`i % 2 == 0`).

6.4 while Loops

A while loop in C++ is a variable-repetition loop that is controlled by a condition. The statements in the body of the loop is executed as long as a specified condition remains true. The syntax for a ‘while’ loop is:

```
while (condition) {
    // statement or block of statements
}
```

Listing 6.7: `while` Loop Syntax

This program reads an undetermined number of marks within the range of 0 to 100 and calculates their sum. The loop terminates when a mark outside this range is entered:

```
#include <iostream>
using namespace std;

int main() {
    int mark, sum = 0;
    cout << "Type_in_a_mark:_";
    cin >> mark;
    while (mark >= 0 && mark <= 100) {
        sum += mark;
        cout << "Type_in_a_mark:_";
        cin >> mark;
    }
    cout << "The_sum_of_the_marks_is:_ " << sum << endl;
    return 0;
}
```

```
}
```

Listing 6.8: while Loop Example

Question 6.3

Why is sum assigned an initial value of 0? What would happen if sum was not assigned an initial value or if it was assigned an initial value of 1?

Question 6.4

Why is it necessary to read a mark before the while-statement and just before the end of the loop's block?

The above questions address a very important principle of the while loop, the so-called i-t-c principle.

- The **i** stands for initialise: before entering a while loop, the condition variable must get an initial value.
- The **t** stands for test: the condition variable is continually tested to check whether we have reached the end of the loop.
- The **c** stands for change: to exit the loop at a stage, the value of the condition variable must change inside the loop (i.e. read or incremented).

The following program facilitates in the playing of the game Number Guess. The first player types in an integer value and the second player must guess the number. The program must keep track of the number of guess attempts the second player has had, and give the player a clue depending on whether the number is too large or too small.

```
#include <iostream>
```

```
using namespace std;
```

```
void displayInstructions() {  
    cout << "Number_Guess_Game\n";  
    cout << "This_is_a_two-player_game.  
    .....The_first_player_looks_away_while_the_first_player  
    .....types_in_a_number.\n";  
    cout << "The_second_player_must_then_guess_the_number.  
    .....Clues_will_be_given!_Good_luck!\n";  
    cout << "Player_1_-_Type_in_the_secret_number:_";  
}
```

```
int main() {  
    int secretNumber, guess, numGuesses = 0;  
    bool correctGuess = false;  
  
    displayInstructions();  
    cin >> secretNumber;  
  
    cout << "Player_2_-_Type_in_your_guess:_";
```

```

    cin >> guess;
    ++numGuesses;

    while (!correctGuess) {
        if (guess > secretNumber) {
            cout << "Your_guess_is_too_high.\n";
        } else if (guess < secretNumber) {
            cout << "Your_guess_is_too_low.\n";
        } else {
            cout << "Your_guess_was_spot_on!\n";
            correctGuess = true;
        }

        if (!correctGuess) {
            cout << "Guess_again:_";
            cin >> guess;
            ++numGuesses;
        }
    }

    cout << "It_took_" << numGuesses << "_guesses_to_guess_the_correct_number\n";
    return 0;
}

```

Listing 6.9: Number Guess Game in C++

The boolean value (`correctGuess`) is used to control the loop. It is assigned the value of false before the while loop and when the guess is correct changed to true within the while loop to terminate the loop.

Question 6.5

Why is it necessary to have the following if-statement in the block of the while loop?

```

if (!correctGuess) {
    cout << "Guess_again:_";
    cin >> guess;
    ++numGuesses;
}

```

Would it have been possible just to write:

```

cout << "Guess_again:_";
cin >> guess;
++numGuesses;

```

without the if-statement? Explain the implications of including or excluding the if-statement within the loop.

6.5 do-while Loops

The do-while loop in C++ is similar to the while loop but guarantees that the body of the loop will be executed at least once because the condition is evaluated at the end of the loop.

```
do {  
    // statements  
} while (condition);
```

Listing 6.10: do-while Loop Syntax

The following program continuously prompts for rainfall values and computes the total until a negative value is entered, which terminates the loop:

```
#include <iostream>  
using namespace std;  
  
int main() {  
    int rainfall, sum = 0, count = 0;  
    do {  
        cout << "Type in a rainfall value (negative value to stop): ";  
        cin >> rainfall;  
        sum += rainfall;  
        count++;  
    } while (rainfall >= 0);  
  
    sum -= rainfall;  
    count -= 1;  
  
    cout << "The average rainfall is: " << (sum*1.0) / count << endl;  
    return 0;  
}
```

Listing 6.11: do-while Example

Question 6.6

What would happen if `sum` was not initialised before the loop?

Question 6.7

Why is `rainfall` subtracted from `sum` and 1 subtracted from `count` after the while loop?

Question 6.8

Refactor the code to avoid subtracting `rainfall` from `sum` and 1 from `count`. You will need a conditional (if) in your while loop.

Question 6.9

What happens when the user immediately enters a negative value? Refactor the code to avoid this behaviour using a ternary if statement.

6.6 Coding Patterns

This section introduces patterns that can be applied when coding. By following these patterns, when presented with the specific scenario they address, your code should work as anticipated and be readable to someone else who may be working on the same code-base as what you are. Each pattern is identified with the keyword Plan, followed by a number, a descriptive title providing the use case for the application of the pattern, a generic code structure and an example where the plan is applied.

For example, the first pattern, Plan 6.A has the descriptive title: Read known number of values / value groups, followed by the general code structure and the example to read in 100 integer values from standard input.

The majority of the patterns listed below relate to reading values from the console (using `cin`) and writing to the console (using `cout`).

Plan 6.A: Read known number of values / value groups

General code structure

```
for (counter = 0; counter < numberOfTimesValue; counter++) {  
    cout << "Enter a value ";  
    cin >> readVariable;  
    ...  
}
```

Example

Read in 100 floating point values from the standard input (keyboard) and determine the square for each value. Write each result to standard output (the screen).

```
#include <iostream>  
  
using namespace std;  
  
int main() {  
    double value;  
  
    for (int counter = 1; counter <= 100; counter++) {  
        cout << "Enter a real number: ";  
        cin >> value;  
        cout << value << " squared is " << (value * value) << endl;  
    }  
    return 0;  
}
```

Note, you could have begin the loop counter at 0. The loop stopping condition would then be specified as `counter < 100`.

Question 6.10

Write a program that reads 5 consecutive integer values from the keyboard and writes

the square root of each of the values to the screen. Make use of the `sqrt` function already defined in `cmath`.

Question 6.11

What happens if a negative value is typed in for one of the values in Question 6.10? Modify the program so that it accepts positive numbers only and displays an error message when a negative value is entered.

Question 6.12

Write a program that reads in 10 pairs of values and determines the sum, difference and product of the values for each iteration of the repetition statement. If you wanted to divide the numbers as well, what would you have to test for?

Plan 6.B: Read number of values first

General code structure

```
cin >> numberOfTimesValue;
for (counter = 0; counter < numberOfTimesValue; counter++) {
    cout << "Enter a value ";
    cin >> readVariable;
    ...
}
```

Example

Read the number of values to be typed in first. Output the square of each subsequent integer value read from the keyboard up to the number of values typed in.

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
    int numberOfValues;
    cout << "How many values are there? ";
    cin >> numberOfValues;

    for (int counter = 0; counter < numberOfValues; ++counter) {
        double value;
        cout << "Enter a real number: ";
        cin >> value;
        cout << value << " " << (value * value) << endl;
    }
    return 0;
}
```

Note the use of `++counter` for the update part of the loop. As the increment is a standalone instruction, either pre- or post-increment can be used.

Question 6.13

Write a program that reads the names and marks for a specific number of students as entered by the user. The program must determine the average mark for all the students.

Note, look at the accumulate plan (Plan 6. F) if you require help to determine the average mark.

Plan 6.C: Read unknown number of values / value groups

General code structure

```
cin >> value;
while (booleanExpression using value) {
    ...
    cin >> value;
}
```

When reading an unknown number of values, it is best to make use of a while-loop. Some programmers do *force* the plan into a for-loop. This is considered bad programming practice.

The value used to exit the loop is referred to as the sentinel.

Example

Read a value, write the value and the square of the value to the screen until the value read is 0.

```
#include <iostream>

using namespace std;

int main() {
    double value;

    cout << "Enter a real number: ";
    cin >> value;

    while (value != 0) {
        cout << value << " " << (value * value) << endl;
        cout << "Enter next number: ";
        cin >> value;
    }

    return 0;
}
```

Question 6.14

What needs to be taken into consideration when the program in the above example is converted to a do-while loop?

Plan 6.D: Read value until valid

General code structure

```
cout << "Enter a value within a specific range";
cin >> value;
```

```

while (booleanExpression using value) {
    cout << "Invalid value, some message about the value";
    cin >> value;
}

```

Example Write a function, `getMark`, to read a test mark until it is valid. A valid mark is any mark between 0 and 100 inclusive.

```

int getMark() {
    int mark;
    cout << "Enter a test mark (between 0 and 100): ";
    cin >> mark;

    while (mark < 0 || mark > 100) {
        cout << "Invalid test mark. Please enter a value between 0
        .....and 100." << endl;
        cout << "Enter a test mark (between 0 and 100): ";
        cin >> mark;
    }
    return mark;
}

```

An example of a main function that calls the `getMark` function is given by:

```

int main() {
    int testMark;

    testMark = getMark();

    if (testMark >= 75) {
        std::cout << "Distinction" << std::endl;
    } else if (testMark >= 50) {
        std::cout << "Pass" << std::endl;
    } else {
        std::cout << "Fail" << std::endl;
    }

    return 0;
}

```

Question 6.15

Write a program that asks the user to first enter the number of values they wish to enter and then reads that number of consecutive values from the keyboard and writes the square root of each of the values to the screen. Make use of the `sqrt` function already defined in the `cmath` library of C++. The pre-condition of the `sqrt` function is that the input parameter must be a positive value. Make sure you test that the pre-condition of the function is not violated.

Note, the square roots of the values typed in MUST be written to the screen. Write a special function that keeps on asking for a number until a positive number has been

entered.

Question 6.16

Write a function, `readUntilValid` that makes use of a do-while statement to determine whether the values entered are valid. Change the program in Question 6.15 to call this function.

Question 6.17

Write a function called `yesNo` that returns true if the user enters *yes* and false if the user enters *no*. The function will repeatedly prompt a user to type in their choice, either *yes* or *no* represented by 'y', 'Y', 'n' or 'N'. The function must keep asking until a valid value is entered (the user could by mistake press 'm' or any other keyboard character). Remember to test for both lower and uppercase letters. Note, you could also use the C++ function `toupper`. Make use of online help to determine which C++ package needs to be included to make use of the function.

Plan 6.E: Count values

General code structure

```
countVariable = 0;
Repetition structure {
    ....
    countVariable++;
}
```

Example

Consider the example given for Plan 6. C. Assume you wish to keep track of the number of values the user entered before they decided to type in the sentinel value of 0. To achieve this, the code in the example will need to change as follows:

```
#include <iostream>

using namespace std;

int main() {
    double value;
    int count = 0;

    cout << "Enter a real number: ";
    cin >> value;

    while (value != 0) {
        cout << value << " " << (value * value) << endl;
        count++;
        cout << "Enter next number: ";
        cin >> value;
    }

    cout << count << " values were entered" << endl;
```

```

    return 0;
}

```

Question 6.18

Adapt the program that reads the names and marks of students (Question 6. 13. The program should read an unknown number of students. That is, rather than asking the user to enter the number of students that are to be typed in, it must read until a specific value is entered. Furthermore, the program must determine the number of students who passed with a mark of 75% or more and also print the names and marks of these students.

Plan 6.F: Accumulate values

General code structure

```

sum = 0;
Repetition structure {
    ...
    cin >> value;
    ...
    sum = sum + value;
}

```

An alternative way to write the statement, `sum = sum + value;` in C++ is `sum += value;`.

Note that for addition the variable `sum` is initialised to the identity element for addition (that is 0). If we wanted to determine the product of a list of numbers, we will need to initialise the variable calculating the product to 1 (the identity element for multiplication).

Question 6.19

Continuing on from the code completed in Question 18, apply the Accumulate values plan and calculate the average mark for all the students that have been entered.

Plan 6.G: Determine largest/smallest value

General code structure

In this general case, we will be determining the largest value.

```

largest = expression;
Repetition structure {
    ...
    value = get value from input stream (e.g. cin);
    if (value > largest) {
        largest = value;
    }
    ...
}

```

The value of `largest` needs to be initialised before it can be compared. The **expression** to which it is to be initialised to must either be the smallest possible value that can be entered, or the first value entered. The strategy followed is dependent on the requirements for which the loop is written. By initialising **largest** to the smallest value before the loop, removes the need to possibly read the value before the loop executes.

Determining the smallest value will follow a similar structure. A variable, for example `smallest` to hold the smallest value needs to be defined and initialised to the largest possible value for the purposes of the program.

Example The following program asks the user to type in an unknown number of positive integer values representing marks and determines the smallest of the values entered. Note the use of a decision statement to write the appropriate message depending on whether the user terminated the program on first iteration or not.

```
#include <iostream>

using namespace std;

int getMark() {
    int mark;

    cout << "Enter a test mark (between 0 and 100, -1 to stop): ";
    cin >> mark;

    while ((mark < 0 || mark > 100) && !(mark == -1)) {
        cout << "Invalid test mark.
        .....Please enter a value between 0 and 100." << endl;
        cout << "Enter a test mark (between 0 and 100): ";
        cin >> mark;
    }
    return mark;
}

int theSmallest(int smallest, int mark) {
    if (mark < smallest) {
        return mark;
    }
    return smallest;
}

int main() {
    int mark;
    int smallest = 101;

    mark = getMark();
    while (mark != -1) {
        smallest = theSmallest(smallest, mark);
        mark = getMark();
    }

    if (smallest == 101) {
        cout << "No mark was entered.";
    } else {
        cout << "The smallest mark entered is: " << smallest << endl;
    }
}
```



```

    return 0;
}

```

Question 6.20

Extend the student marks example to determine the student who did the best and the student who did the worst. The program must print the name of the student with the highest mark as well as their mark to the screen. Similarly, the program must print the name of the student with the lowest mark along with the mark they received.

Plan 6.H: Swop (or swap) two values

General code structure

Assume two values, `value1` and `value2`, are given and that these values are type compatible. A third variable, say `temp` is declared and the values ‘in’ `value1` and `value2` are swapped using this third variable as follows:

```

temp = value1;
value1 = value2;
value2 = temp;

```

Swapping values works best when written as a function. We will revisit this when we discuss pass parameters by reference to a function.

Question 6.21

Write a program that accepts two values as input. The program swops the values only if the first value is greater than the second value. For example: an input of 10 30 will output 10 30, while an input of 30 10 will output 10 30.

Question 6.22

Write a program that reads the names and marks of an unknown number of students. The program must determine the average mark as well as the names and marks of the students who achieved the highest and second-highest marks. Refer to the program you have previously written that form part of this example.

Plan 6.I: Search for value

General code structure

```

conditionVariable = false;
while ((logicalExpression) && !(conditionVariable)) {
    ....
    if (someVariable = someExpression) {
        conditionVariable = true;
    }
}

```

On most occasions, when the loop terminates, a test needs to be conducted whether the `conditionVariable` resulted in the loop terminating or whether the `logicalExpression` caused the loop to terminate.

With this plan, while the user is entering values, a specific value is being looked out for which can influence the stopping condition of the loop. This plan is generally written

using a while loop.

Question 6.23

Identify the plan in the NumberGuess program given Listing 6.9.

Question 6.24

Change the NumberGuess program (Listing 6.9) so that it makes use of the function written in Question 6.17 to govern an extra loop that gives the user the option to play the game again.

6.7 Translation between Repetition Statements

Translating between different types of loops (for, while, and do-while) involves understanding the control structure of each loop and ensuring that the conversion maintains the same logic and flow of the original loop.

6.7.1 Translating for Loops to while and do-while Loops

A ‘for’ loop in C++ is typically used for iterations where the number of repetitions is known before the loop starts. Its structure inherently includes initialisation, condition checking, and updating expressions all in one line. A **for** loop can be written in terms of a **while** or **do-while** loop.

6.7.1.1 Using a while Loop

A for loop:

```
for (int i = 1; i <= 10; ++i) {  
    cout << i << endl;  
}
```

Listing 6.12: for Loop Example

Can be translated into a while loop as follows:

```
int i = 1;  
while (i <= 10) {  
    cout << i << endl;  
    i++;  
}
```

Listing 6.13: Equivalent **while** Loop

Here, the initialisation occurs before the loop, the condition is checked at the beginning of each iteration, and the index is incremented inside the loop body.

6.7.1.2 Using a do-while Loop

Similarly, translating to a do-while loop ensures the body of the loop executes at least once:

```

int i = 1;
do {
    cout << i << endl;
    i++;
} while (i <= 10);

```

Listing 6.14: Equivalent do-While Loop

Question 6.25

Translate Listing 6.3 to a while loop and do while loop.

Question 6.26

Translate Listing 6.4 to a while loop and do while loop

6.7.2 Translating while and do-while Loops to for Loops

It is only possible to write ‘while’ and ‘do-while’ loops in terms of a ‘for’ loops where you know in advance how many times these loops must be executed. Therefore, these loops should have conditions governed by a loop index, that is updated consistently.

6.7.2.1 From while Loop

Given a ‘while’ loop:

```

int i = 0;
while (i < 5) {
    cout << "Number:_" << i << endl;
    i++;
}

```

Listing 6.15: While Loop Example

A for loop representation would be:

```

for (int i = 0; i < 5; i++) {
    cout << "Number:_" << i << endl;
}

```

Listing 6.16: Translated For Loop

6.7.2.2 From do-while Loop

Translating a do-while loop that executes at least once might require additional considerations, especially if the condition affects whether the loop should continue after the first iteration:

```

int i = 0;
do {
    cout << "Number:_" << i << endl;
    i++;
} while (i < 5);

```

Listing 6.17: Do-While Loop Example

This can be directly translated into a ‘for’ loop as it doesn’t depend on conditions that might change externally before the loop starts.

Question 6.27

Is it possible to write Listing 6.8 in terms of a for loop. Why, or why not?

Question 6.28

Translate the following while loop to a for loop:

```
int i = 1;
int j = 10;
while(i < j){
    cout << i << " " << j << endl;
    i++;
    j--;
}
```

Listing 6.18: while loop to be translated

Remember that a for loop can have multiple loop control variables and updates.

Question 6.29

Translate the following do-while loop to a for loop:

```
int i = 64;
do {
    cout << "Number: " << i << endl;
    i = i / 2;
} while (i > 1);
```

Listing 6.19: do-while loop to be translated

When translating between these loops, ensure that side effects in condition evaluations or updates are preserved, particularly if they involve function calls or complex expressions. Sometimes, a direct translation isn’t feasible without restructuring the logic, especially when using external conditions in while and do-while loops.

6.8 Good programming practices

In this section we will consider two aspects of loops that should be used with caution. These being, writing infinite loops and the use of the break and continue statements. It is therefore good programming practice not to use these constructs. If you however need to use them, this section will enable you to understand how and when to apply them and what the pitfalls and shortfalls are when using them. The section concludes with a technique, similar to using pre- and post-conditions for verifying function correctness. This technique to verify the correctness of loops is referred to as loop invariants. Loop invariants can be tested using assert statements within the code.

6.8.1 Infinite Loops

Infinite loops are loops that continue to run as there is no stopping condition coded for the loop. In many cases, loops run on forever due to programming errors. This may be

as a result of incorrect terminating conditions or not incrementing the loop variable in order for the termination condition to be met at some stage. Uncontrolled infinite loops may result in unexpected side-effects or even the program crashing. Infinite loops can use precious system resources.

There three types of infinite loops. Loops that look infinite, but due to some exiting statement (usually a **break**) somewhere within their body, they do terminate. Loops that are intended to continue indefinitely, and loops that due to some programming error should not continue executing.

6.8.1.1 Loops that look like they are infinite

Loops that look infinite and can be exited with a break statement, are the most dangerous type of infinite loop. In many cases, the outcome of the loop is not what is expected. This is especially true when the code base becomes large and the exiting statement is lost between lines of code. The following is a very simple example in which any value entered that is greater than 20000 will result in the loop terminating.

```
value = 0;
while (true) {
    if (value > 20000) break;
    cin >> value;
}
```

6.8.1.2 Loops not intended to be infinite

When coding it is possible to inadvertently create an infinite loop. Consider the following code:

```
int i = 0;
while (i < 1000)
    cin >> y;
    i++;
```

In this example, the variable, *i*, that forms part of the condition is not part of the while loops statement block. As *i* is never updated, this code will result in an unintentional infinite loop.

Consider the following code that contains an infinite for loop in the main function.

```
#include <iostream>

using namespace std;

void loopTest(int j) {
    for (int i = 0; i < j ; i++) {
        cout << i << " ";
    }
}

int main() {
```

```

    for (int i = 1; ; i++) {
        cout << i << " ";
        loopTest(i);
    }
    return 0;
}

```

Listing 6.20: Infinite for loop

When will the for loop terminate? There is no condition to test against and therefore the loop will continue to execute for a long time or until the operating system decides to terminate it due to it taking up too much system time or resources.

6.8.1.3 Intentional infinite loops

There is a place for infinite loops.

In many of these cases the loops do have a mechanism that allows them to terminate, such as an event that occurs that is being monitored by the program or a conditional that is set and tests and results in the program breaking out of the loop. Coding an infinite loop under these circumstances is done when very specific functionality is required that an ordinary loop with a terminating condition will not work.

Intentional infinite loops are used in specific applications, such as game loops, networking client-server based application such as web-servers, and real-time data streaming, to name a few. The program never terminates on its own, it is forced by the operating system to stop executing when the process that is running is killed.

Examples of intentional infinite loops include loops where the condition is always true and there is no mechanism coded to exit the loops. For example:

```

for (;;) {
    ...
}

```

or

```

while (true) {
    ...
}

```

6.8.2 Using break and continue

The **break** statement is used to exit the loop when the condition is met. The input statement is skipped and the processing continues after the while loop.

A **continue** statement skips the current iteration of the loop and continues to the next iteration of the loop. For example, the following code will print only the even numbers. Iterations of the loop for the odd values of **i** will be forced past the **cout** statement following the if-statement.

```

for (int i = 1; i < 20; i++) {
    if (i % 2 != 0) continue;
    cout << i << " is even" << endl;
}

```

Listing 6.21: Example code using `continue`

Question 6.30

To appreciate the difference between a `continue` and a `break` statement, write a main function using the code give in Listing 6.21 as the body of the main function. Compile and run the code and observe what the output is before replacing the `continue` statement with a `break` statement.

A word of warning. It is never good programming to intervene with the logic of the code. We have well-formed constructs for a reason. Using these well-formed constructs will result in verifiable and predictable implementations. The code given in Listing 6.21 can be rewritten as follows resulting is a solution that is easier to read, understand, maintain and verify.

```

for (int i = 1; i < 20; i++) {
    if (i % 2 != 0) {
    } else {
        cout << i << " is even" << endl;
    }
}

```

Listing 6.22: Example code equivalent to Listing 6.21

By applying the condition swapping of if-statements discussed in Study Unit 5, this if-statement can be further simplified to:

```

for (int i = 1; i < 20; i++) {
    if (i % 2 == 0) {
        cout << i << " is even" << endl;
    }
}

```

6.8.3 Loop Invariant

Where pre- and post-conditions are used specifically for documenting what must be true before a function is called and what must be true after the function has completed its execution, loop invariants are used to document the conditions under which a loop will execute as expected. Loop invariants, can be tested using `assert` statements, introduced in Study Unit 5.

A loop invariant is a formal statement about the relationship between variables that must be true before the loop starts, before each iteration of the loop starts and directly after the loop has completed execution. Note, a loop invariant may be false during the execution of the body of the loop.

A loop invariant is used to reason about the correctness of the loop using formal method techniques and should not be confused with the loop condition.

Consider a program where the user enters a positive integer value and the program calculates the sum of all positive integers up to and including the value entered by the user.

```
#include <iostream>

using namespace std;

int main() {
    int sum = 0;
    int i = 0;
    int n;

    cout << "Enter a positive integer n: ";
    cin >> n;

    for (i = 1; i <= n; ++i) {
        sum += i;
    }

    cout << "The sum of the first " << n
         << " positive integers is: " << sum << endl;

    return 0;
}
```

Listing 6.23: Sum the First n Positive Integer Values

Before the loop executes, the `sum` is 0, `i` is 0 and the value of `n` is the number that the user has typed in. We can therefore say that `sum` contains the sum of first zero positive integers. After one iteration of the loop, `sum` is 1 and `i` is 1, therefore `sum` contains the sum of the first positive integer. Each iteration thereafter will be similar till the loop terminates, when `sum` contains the sum of the first `n` positive integers. The loop invariant can be generalised to state that: `sum` contains the sum of the first `i` positive integers.

To test the loop invariant using the `assert` statement we need to determine an alternative means of calculating the sum of the first `i` positive integers. The formula to do just this is given by $i * (i + 1) / 2$. We can therefore add `assert` statements to the code given in Listing 6.23 that tests if `sum == i * (i + 1) / 2`.

```
#include <iostream>
#include <cassert>

using namespace std;

int main() {
    int sum = 0;
    int i = 0;
    int n;
```



```

    cout << "Enter a positive integer n: ";
    cin >> n;

    assert(sum == i * (i + 1) / 2);
    for (i = 1; i <= n; ++i) {
        sum += i;
        assert(sum == i * (i + 1) / 2);
    }
    assert(sum == (i - 1) * (i) / 2);
    cout << "The sum of the first " << n
        << " positive integers is: " << sum << endl;

    return 0;
}

```

Listing 6.24: Sum the First n Positive Integer Values

Note the formula change after the loop. This is necessary because the value of `i` is increased by 1 after the loop condition has been checked. So we the value of $i - 1$, which when substituted into the formula for `i` becomes $(i - 1) * (i) / 2$.

Using loop invariant assertions when coding is a good beginning to check and test code. There are better testing techniques when taking code from development to production.

6.9 Document Change Log

Date	Change
3 May 2024	Initial preparation and presentation of the document.